

Массивы и методы **map**, **filter**, **find**

Исчерпывающий разбор функциональных методов высшего порядка, алгоритмов декларативной трансформации коллекций, предикатных функций и правил генерации списков в React.

 zsh — array pipeline preview

```
// Декларативный пайплайн обработки коллекции в одну
// строку:
const activeTitles = users .filter(user => user.isActive) .map(user
=> user.name);

// Исходный массив остался абсолютно нетронутым
// (Иммутабельность)!
```

Анатомия массивов в JS и функции высшего порядка

🔧 1. Устройство массивов в V8: Fast Elements vs Dictionary Mode

В отличие от языков низкого уровня (C++, Rust), где массив — это непрерывная область памяти фиксированного размера с элементами одного типа, в JavaScript массив является **специализированным объектом**, ключами которого выступают строковые индексы `"0"`, `"1"`, `"2"`.

Движок Google V8 оптимизирует массивы в памяти на два внутренних режима:

- **Fast Elements (Плотные массивы):** Элементы хранятся в непрерывном буфере памяти. Доступ по индексу `0(1)`. Этот режим активен, если массив не содержит «дыр» (пустых индексов) и хранит однородные типы данных.
- **Dictionary Elements (Разреженные массивы / Holey Arrays):** Если в массиве появляются пропуски (например, `arr[1000] = 5` при длине 2), V8 переводит массив в хэш-таблицу. Память экономится, но скорость доступа падает.

2. Функции высшего порядка (Higher-Order Functions) и колбэки

Функция называется **функцией высшего порядка** (High-Order Function, HOF), если она удовлетворяет хотя бы одному условию:

1. Принимает другую функцию в качестве аргумента (такая функция называется *колбэком* / *callback*).
2. Возвращает новую функцию в качестве результата своего выполнения.

Методы `map`, `filter`, `find` являются классическими HOF. Они абстрагируют управление циклами и счетчиками, позволяя разработчику сосредоточиться исключительно на бизнес-правиле обработки каждого конкретного элемента.



Императивный цикл for (Антипаттерн)

Разработчик вручную инициализирует счетчик `let i = 0`, следит за границей `i < arr.length` и мутрует внешний результирующий массив. Высокий риск ошибки «на единицу» (off-by-one error).



Декларативный метод HOF (Стандарт React)

Разработчик декларирует *что* нужно сделать с каждым элементом через чистую функцию. Исходная коллекция не мутруется, код лаконичен и предсказуем.

Метод `.map()`: Декларативная проекция данных

🔗 1. Концепция и математическая сигнатура метода

Метод `.map()` производит поэлементную проекцию коллекции. Он вызывает переданный колбэк для каждого элемента исходного массива и возвращает **абсолютно новый массив точно такой же длины** ($\mathbb{N} \rightarrow \mathbb{N}$).

ИСХОДНЫЙ МАССИВ (A)

ДЛИНА: 3

`[ { price: 100 }, { price: 200 }, { price: 300 } ]`



НОВЫЙ МАССИВ (B) = MAP()

ДЛИНА: 3

`[ "$100.00", "$200.00", "$300.00" ]`

Полная сигнатура колбэка метода `.map()`:

```
const newArray = array.map((element, index, array) => {  
  // element — текущий обрабатываемый элемент  
  // index — порядковый номер (индекс) элемента (0, 1, 2...)   
  // array — ссылка на исходный массив  
  return transformedElement; });
```

2. Применение .map() в React: Генерация JSX списков

В React метод `.map()` является единственным стандартным способом рендеринга списков компонентов. JSX не может принимать классические циклы, но идеально принимает массивы элементов:

```
function
ProductList({ products }) {
  return ( <ul
    className="product-grid"> {products.map(item => (
      // Атрибут key ОБЯЗАТЕЛЕН на самом внешнем теге внутри map:
      <li
        key={item.id}
        className="product-card"> <h4>{item.title}</h4> <span>Цена: {item.price}
        &lt;/span> </li> )}} </ul> ); }
```

⚠ Почему нельзя использовать index массива как key?

Если порядок элементов в списке может меняться (удаление, добавление в начало, сортировка), использование `key={index}` разрушает алгоритм согласования React (Diffing). React сопоставит узлы по номерам индексов, что приведет к графическим багам в полях ввода и лишним ререндерам. Всегда используйте стабильный `item.id`.

Метод `.filter()`: Предикатная фильтрация

1. Предикаты и длина результирующего массива

Метод `.filter()` вызывает переданную функцию-предикат для каждого элемента. **Предикат** — это функция, возвращающая логическое значение: `true` (элемент включается в новый массив) или `false` (элемент отбрасывается).

Длина результирующего массива всегда лежит в диапазоне от 0 до исходной длины ($0 \leq M \leq N$):

ИСХОДНЫЙ МАССИВ (N = 4)

```
[ { age: 16 }, { age: 22 }, { age: 15 }, { age: 30 } ]
```



`FILTER(U  $\Rightarrow$  U.AGE  $\geq$  18)`

M = 2

```
[ { age: 22 }, { age: 30 } ]
```

2. Удаление элементов из React State через .filter()

В React удаление записи из стейта реализуется фильтрацией массива по принципу «оставить все элементы, чей ID не равен удаляемому»:

```
deletePattern.jsx

const [tasks, setTasks] =
  useState([ { id:
    "t1", text:
    "Выучить JS" }, { id:
    "t2", text:
    "Понять React" }, { id:
    "t3", text:
    "Сдать проект" } ]);

function
handleDelete(taskIdToDelete) {
  // Создаем новый массив БЕЗ элемента с указанным id:
  setTasks(prevTasks => prevTasks.filter(task => task.id !== taskIdToDelete)); }
```

Методы поиска: `.find()`, `.findIndex()`, `.some()`, `.every()`

🎯 1. Метод `.find()` и ранний выход (Early Exit)

В отличие от `.filter()`, который всегда проверяет массив до самого конца, метод `.find()` возвращает **первый найденный элемент**, удовлетворяющий условию предиката, и **мгновенно прекращает дальнейший обход коллекции**.

Если элемент не найден, `.find()` возвращает `undefined`.

Метод	Что возвращает	Если не найдено	Сложность
<code>.find()</code>	Первый совпавший элемент коллекции	<code>undefined</code>	$\backslash(O(1))$ в лучшем / $\backslash(O(N))$ в худшем
<code>.findIndex()</code>	Числовой индекс первого совпавшего элемента	<code>-1</code>	$\backslash(O(1))$ в лучшем / $\backslash(O(N))$ в худшем
<code>.some()</code>	<code>true</code> , если хотя бы 1 элемент прошел проверку	<code>false</code>	$\backslash(O(1))$ при раннем нахождении
<code>.every()</code>	<code>true</code> , если ВСЕ элементы удовлетворяют условию	<code>false</code>	$\backslash(O(1))$ при первом же несовпадении

```
const users = [ { id:
101, name:
"Анна", role:
"admin" }, { id:
102, name:
"Борис", role:
"user" } ];

// 1. Поиск конкретного объекта:
const currentAdmin = users.find(u => u.role === "admin");
console.log(currentAdmin?.name);
// "Анна"

// 2. Быстрая проверка прав доступа:
const hasAdminRights = users.some(u => u.role === "admin");
// true
```

Композиция методов (Method Chaining) и производительность

1. Конвейерная обработка (Chaining)

Поскольку методы `.filter()` и `.map()` возвращают новые массивы, их можно связывать в непрерывные цепочки обработки данных (Data Processing Pipelines):

```
const rawProducts = [ { id:
1, name:
"Клавиатура", price:
4000, inStock:
true }, { id:
2, name:
"Монитор", price:
25000, inStock:
false }, { id:
3, name:
"Мышь", price:
2000, inStock:
true } ];
```

```
// Задача: Отфильтровать товары в наличии, применить скидку 10% и
отформатировать:
```

```
const formattedSaleItems =
rawProducts .filter(product =>
product.inStock) .map(product => ({ id:
product.id, name: product.name, salePrice: product.price *
0.9 })); console.log(formattedSaleItems);
```

🧠 2. Алгоритмическая сложность и оптимизация аллокации памяти

При связывании цепочки методов `arr.filter().map()` массив обходится дважды, создавая промежуточный массив в куче (Heap Memory). В 99% сценариев React на коллекциях до 10 000 элементов это не создает проблем производительности.

🕒 Мемоизация цепочек в React через `useMemo`

Если вы фильтруете и сортируете большой список данных на клиенте, всегда оборачивайте конвейер трансформации в хук `useMemo`, чтобы тяжелые вычисления не выполнялись повторно при каждом постороннем ререндере компонента:

```
const visibleProducts = useMemo(() => {  
  return products.filter(p => p.price ≤ maxPrice).map(formatProduct);  
}, [products, maxPrice]);
```

Сводная матрица методов массивов

Метод	Мутирует оригинал?	Результат работы	Основной сценарий в React
<code>.map()</code>	НЕТ (Чистый)	Новый массив той же длины ($\backslash (N \backslash \text{to } N \backslash)$)	Рендеринг списков JSX <code><li key= { ... } ></code> .
<code>.filter()</code>	НЕТ (Чистый)	Новый массив меньшей/равной длины	Удаление по ID, живой поиск и фильтры.
<code>.find()</code>	НЕТ (Чистый)	Один найденный элемент или <code>undefined</code>	Поиск выбранной сущности по ID в стейте.
<code>.reduce()</code>	НЕТ (Чистый)	Любое свернутое значение (число, объект)	Подсчет корзины, группировка по категориям.
<code>.push()</code> / <code>.splice()</code>	ДА (Мутирует!)	Изменяет исходный массив в памяти	ЗАПРЕЩЕНО использовать в React State!